

Bài tập

1. Lập trình hướng đối tượng (OOP)

a) Khái niệm:

Lập trình hướng đối tượng là một mô hình lập trình tập trung vào các "đối tượng" và cách chúng tương tác với nhau. Trong OOP, mỗi đối tượng đại diện cho một thực thể trong thế giới thực với các thuộc tính (dữ liệu) và phương thức (hành vi). Sử dụng lớp và đối tượng để mô phỏng các thực thể và mối quan hệ trong thế giới thực. Nó giúp tạo ra các chương trình dễ bảo trì, dễ mở rộng và có khả năng tái sử dụng mã cao.

Các nguyên tắc cơ bản đằng sau việc sử dụng các kỹ thuật lập trình hướng đối tượng bao gồm:

- Đối tượng (object).
- Lớp (class).
- Thừa kế (Inheritance).
- Đa hình (polymorphism).

b) Ví dụ minh họa:

- ✓ Lập trình game: Các nhân vật, vật phẩm và môi trường trong game đều có thể được mô phỏng bằng các đối tượng.
- ✓ Hệ thống quản lý nhân viên: Mỗi nhân viên có thể được đại diện bởi một đối tượng với các thuộc tính như tên, tuổi, vị trí và mức lương.
- ✓ Hệ thống quản lý kho hàng: Mỗi sản phẩm có thể được đại diện bởi một đối tượng với các thuộc tính như tên, mô tả, giá cả và số lượng tồn kho.
- ✓ VD code: Chương trình tính diện tích và chu vi hình tròn.

```
#include isostream
```

```
using namespace std;
```

```
class HìnhTron {
```

```
private:
```

```
    float banKinh;
```

```
public:
```

```
    HìnhTron(float banKinh) {
```

```
        this->banKinh = banKinh;
```

```
    }
```

```
    float tinhChuVi() {
```

```
        return 2 * 3.14 * banKinh;
```

```
    }
```

```
    float tinhDienTich() {
```

```
        return 3.14 * banKinh * banKinh;
```

```
    }
```

```
};
```

```
HìnhTron hìnhTron1(5); // Tạo đối tượng hìnhTron1 từ lớp HìnhTron
```

```
float chuVi1 = hìnhTron1.tinhChuVi();
```

```
float dienTich1 = hìnhTron1.tinhDienTich();
```

```
cout << "Chu vi hìnhTron1: " << chuVi1 << endl;
```

```
cout << "Dien tich hìnhTron1: " << dienTich1 << endl;
```

```
class HìnhHoc {
private:
    float chuVi;
    float dienTich;
public:
    virtual void tinhChuVi() = 0; // Phương thức ảo, cần được định nghĩa lại trong lớp con
    virtual void tinhDienTich() = 0; // Phương thức ảo, cần được định nghĩa lại trong lớp con
};

class HìnhTron : public HìnhHoc {
public:
    HìnhTron(float banKinh) {
        this->banKinh = banKinh;
    }
    void tinhChuVi() override {
        chuVi = 2 * 3.14 * banKinh;
    }
    void tinhDienTich() override {
        dienTich = 3.14 * banKinh * banKinh;
    }
};

class HìnhHoc {
private:
    float chuVi;
    float dienTich;
public:
    void inThongTin() {
        cout << "Chu vi: " << chuVi << endl;
        cout << "Dien tich: " << dienTich << endl;
    }
};

class HìnhTron : public HìnhHoc {
public:
    HìnhTron(float banKinh) {
        this->banKinh = banKinh;
    }
    void tinhChuVi() override {
        chuVi = 2 * 3.14 * banKinh;
    }
    void tinhDienTich() override {
        dienTich = 3.14 * banKinh * banKinh;
    }
};
```

```
}  
void inThongTin() override {  
    HìnhHoc::inThongTin(); // Gọi phương thức inThongTin() của lớp cha  
    cout << "Ban kinh: " << banKinh << endl;  
}  
};  
int main() {  
    HìnhTron hìnhTron1(5);  
    hìnhTron1.tinhChuVi();  
    hìnhTron1.tinhDienTich();  
    hìnhTron1.inThongTin();  
    return 0;  
}
```

2. Lớp và đối tượng (class & object)

a) Lớp (Class)

i. Khái niệm:

Là một tập các đối tượng cùng loại, đại diện cho kiểu dữ liệu trừu tượng. Được sử dụng như kiểu dữ liệu do người dùng định nghĩa. Là bản mẫu để tạo ra thể hiện của đối tượng. Nó giúp tổ chức code hiệu quả, tái sử dụng code dễ dàng và tăng tính bảo trì cho chương trình.

ii. Ví dụ minh họa:

```
class HìnhTron {  
private:  
    float banKinh;  
public:  
    HìnhTron(float banKinh) {  
        this->banKinh = banKinh;  
    }  
    float tinhChuVi() {  
        return 2 * 3.14 * banKinh;  
    }  
    float tinhDienTich() {  
        return 3.14 * banKinh * banKinh;  
    }  
};
```

b) Đối tượng (object)

i. Khái niệm:

Đại diện cho một thực thể trong thế giới thực với các thuộc tính và phương thức. Các đối tượng là thể hiện của cùng 1 lớp có dữ liệu ở trạng thái khác nhau.

OBJECT = DATA (attributes) + METHODS (behavior)

ii. Ví dụ minh họa:

```
HìnhTron hìnhTron1(5); // Tạo đối tượng hìnhTron1 từ lớp HìnhTron
```

```
float chuVi1 = hìnhTron1.tinhChuVi();  
float dienTich1 = hìnhTron1.tinhDienTich();  
cout << "Chu vi hìnhTron1: " << chuVi1 << endl;  
cout << "Dien tich hìnhTron1: " << dienTich1 << endl;
```

3. Kế thừa (Inheritance)

a) Khái niệm

Cho phép một lớp (lớp con) "thừa hưởng" các thuộc tính và phương thức của một lớp khác (lớp cha). Cung cấp cơ chế rất mạnh trong việc tổ chức và cấu trúc chương trình. Giúp tái sử dụng mã và tạo ra các lớp mới từ các lớp hiện có, có thể cài đặt lớp cha là lớp trừu tượng. Có 2 loại kế thừa:

- Đơn kế thừa
- Đa kế thừa

b) Ví dụ minh họa:

- ✓ Trong ngành ô tô, các mẫu xe mới kế thừa thiết kế từ các mẫu cũ, chỉ thay đổi một vài chi tiết.
- ✓ Trong hình học, hình lập phương (Cube) kế thừa từ hình vuông (Square), chỉ bổ sung thêm thuộc tính chiều sâu.
- ✓ Trong lập trình, lớp DisposableDish có thể kế thừa từ lớp Dish và thêm các thuộc tính đặc biệt cho đồ dùng một lần.
- ✓ VD code:

```
class HìnhHoc {  
private:  
    float chuVi;  
    float dienTich;  
public:  
    virtual void tinhChuVi() = 0; // Phương thức ảo, cần được định nghĩa lại trong lớp con  
    virtual void tinhDienTich() = 0; // Phương thức ảo, cần được định nghĩa lại trong lớp con  
};  
class HìnhTron : public HìnhHoc {  
public:  
    HìnhTron(float banKinh) {  
        this->banKinh = banKinh;  
    }  
    void tinhChuVi() override {  
        chuVi = 2 * 3.14 * banKinh;  
    }  
    void tinhDienTich() override {  
        dienTich = 3.14 * banKinh * banKinh;  
    }  
};
```

4. Đa hình (polymorphism)

a) Khái niệm:

Hiểu nôm na nó “Đa hình” là “Nhiều hình thức”. Cho phép một phương thức cùng tên hoạt động khác nhau đối với các đối tượng, lớp khác nhau. Giúp code linh hoạt và dễ dàng mở rộng. Được thực hiện bởi quá tải và nạp chồng.

b) Ví dụ minh họa:

- ✓ Trong toán học, phép nhân có thể nhân hai số nguyên, ba số thực,...
- ✓ Trong thực tế, bạn dùng các phương pháp khác nhau để "Đi (drive)" tùy theo ngữ cảnh: lái xe, đánh golf, đưa ra quan điểm.
- ✓ Trong lập trình hướng đối tượng, phương thức clean có thể hoạt động khác nhau trên các đối tượng Dish, Car, Baby.
- ✓ VD code

```
class HìnhHoc {  
private:  
    float chuVi;  
    float dienTich;  
public:  
    void inThongTin() {  
        cout << "Chu vi: " << chuVi << endl;  
        cout << "Dien tich: " << dienTich << endl;  
    }  
};  
  
class HìnhTron : public HìnhHoc {  
public:  
    HìnhTron(float banKinh) {  
        this->banKinh = banKinh;  
    }  
    void tinhChuVi() override {  
        chuVi = 2 * 3.14 * banKinh;  
    }  
    void tinhDienTich() override {  
        dienTich = 3.14 * banKinh * banKinh;  
    }  
    void inThongTin() override {  
        HìnhHoc::inThongTin(); // Gọi phương thức inThongTin() của lớp cha  
        cout << "Ban kinh: " << banKinh << endl;  
    }  
};
```
